

MIT

**Academy of
Engineering**

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

EDS

CodeTantra Codes

Name: Vedant Deshmukh

PRN: 202501040046

Division: CS1

Batch: C13

Subject: Essentials of Data Science

Practical 4

4.1. Practice Lab Assignments

4.1.1. Pandas – Series Creation and Manipulation

4.1.2. Dictionary to DataFrame

4.1.3. Student Information

4.2. Lab Assignments

4.2.1. Month with the Highest Total Sales

4.2.2. Best Selling Product

4.2.3. City that Sold the Most Products

4.2.4. Most Frequently Sold Product Pairs

4.2.5. Titanic Dataset Analysis and Data Cleaning

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

4.1.1. Pandas - series creation and manipulation

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

seriesMa...

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 series=pd.Series(numbers)
8 # Grouping by even and odd numbers and calculating the mean
9 grouped = series.groupby(series % 2==0).mean()
10
11 # Display the mean of even and odd numbers with labels
12 grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
13 print("Mean of even and odd numbers:")
14 print(grouped)
15
```

Average time 0.005 s 5.50 ms Maximum time 0.014 s 14.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 14 ms

Expected output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:0

Odd5.00

Even6.00

dtype: float64

Actual output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:0

Odd5.00

Even6.00

dtype: float64

Terminal Test cases

< Prev Reset Submit Next >

4.1.2. Dictionary to dataframe

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

Sample Test Cases

datafram...

```
94
95 # Display the DataFrame after modifying a row
96 print("After modifying a row:")
97 print(df)
98
99 # Deleting a row
100 n=int(input("Index of row to delete:"))
101 df=df.drop(index=n).reset_index(drop=True)
102
103
104 # Display the DataFrame after deleting a row
105 print("After deleting a row:")
106 print(df)
107
108 # Adding a new column
109 l=input("Enter genders separated by space: ").split()
110 d1={
111     'Gender':l
112 }
113 df['Gender']=l
114
115
116 # Display the DataFrame after adding a new column
117 print("After adding a new column:")
118 print(df)
119
120 # Modifying a column
```

Average time 0.041 s 40.50 ms Maximum time 0.049 s 49.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 49 ms

Expected output

Original DataFrame:0

.....Name Age

0.....Alice...25

1.....Bob...30

2.....Charlie...35

New name: Susan

Actual output

Original DataFrame:0

.....Name Age

0.....Alice...25

1.....Bob...30

2.....Charlie...35

New name: Susan

Terminal Test cases

< Prev Reset Submit Next >

4.1.3. Student Information

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks.

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

Sample Test Cases

student...

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
6
7
8 print("First five rows:")
9 print(data.head())
10
11 average_age=round(data["Age"].mean(),2)
12 print(f"Average age: {average_age}")
13
14 filtered_students=data[data["Grade"]<="B"]
15 print("Students with a grade up to B")
16 print(filtered_students)
```

Average time

0.023 s

23.00 ms

Maximum time

0.034 s

34.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1

Expected output

studentdata.txt

First five rows:

... Name Age Grade

0 John 25 A

1 Alin 22 B

2 Emma 24 A

Actual output

studentdata.txt

First five rows:

... Name Age Grade

0 John 25 A

1 Alin 22 B

2 Emma 24 A

Terminal

Test cases

< Prev Reset Submit Next >

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # Convert 'Date' column to datetime
10 df['Date'] = pd.to_datetime(df['Date'])
11
12 # Extract 'Year-Month' for grouping
13 df['Month'] = df['Date'].dt.to_period('M')
14
15 # Calculate Total Sales for each row (Quantity * Price)
16 df['Total_Sales'] = df['Quantity'] * df['Price']
17
18 # Group by 'Month' and sum the total sales
19 monthly_sales = df.groupby('Month')['Total_Sales'].sum()
20 # Find the month with the highest total sales
21 best_month = monthly_sales.idxmax()
22 highest_sales = monthly_sales.max()
23
24 print(f"Best month: {best_month}")
25 print(f"Total sales: ${highest_sales:.2f}")
26
```

Average time

0.015 s

15.33 ms

Maximum time

0.032 s

32.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

Expected output

sales_data.csv

Best month: 2025-01

Total sales: \$1210.00

Actual output

sales_data.csv

Best month: 2025-01

Total sales: \$1210.00

Terminal

Test cases

< Prev Reset Submit Next >

CODETANTRA

HomeLearn Anywhere

202501040046@mitaoe.ac.inSupportLogout

4.2.2. Best Selling Product

0.032

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.

Find the product that sold the most in terms of quantity sold.

Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	6	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

1import pandas as pd

2

3# Prompt the user for the file name

4file_name = input()

5

6# Load the data

7df = pd.read_csv(file_name)

8

9product_sales=df.groupby("Product")["Quantity"].sum()

10# Find the product with the highest total quantity sold

11best_product = product_sales.idxmax()

12highest_quantity = product_sales.max()

13

14# Display the result

15print(f"Best selling product: {best_product}")

16print(f"Total quantity sold: {highest_quantity}")

17

Average time0.013 s13.33 msMaximum time0.028 s28.00 ms1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1Expected outputActual output

sales_data.csvsales_data.csv

Best selling product: Product ABest selling product: Product A

Total quantity sold: 23Total quantity sold: 23

TerminalTest cases

CODETANTRA

HomeLearn Anywhere

202501040046@mitaoe.ac.inSupportLogout

4.2.3. City that Sold the Most Products

0.032

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the columns: Date, Product, Quantity, Price, and City.

Group the data by city and calculate the total quantity of products sold for each city.

Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	6	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

1import pandas as pd

2

3# Prompt the user for the file name

4file_name = input()

5

6# Load the data

7df = pd.read_csv(file_name)

8

9city_sales=df.groupby("City")["Quantity"].sum()

10

11# write the code..

12best_city=city_sales.idxmax()

13

14# Display the result

15print(f"City sold the most products: {best_city}")

16

Average time0.011 s11.00 msMaximum time0.024 s24.00 ms1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1Expected outputActual output

sales_data.csvsales_data.csv

City sold the most products: Los AngelesCity sold the most products: Los Angeles

TerminalTest cases

CODETANTRA

HomeLearn Anywhere

202501040046@mitaoe.ac.inSupportLogout

4.2.4. Most Frequently Sold Product Pairs

0.045

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

The CSV file contains the following columns: Date, Product, Quantity, Price, and City.

For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).

Output the product pairs that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	6	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanation:
Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- Product A and Product B: Appear in transactions on 2025-01-01 and 2025-01-03.
- Product A and Product C: Appear in transactions on 2025-01-02 and 2025-01-05.
- Product B and Product C: Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- Product A and Product B (2 times)
- Product A and Product C (2 times)

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

frequent...

1import pandas as pd

2from itertools import combinations

3from collections import Counter

4

5# Prompt user to input the file name

6file_name = input()

7

8# Read data from the specified CSV file

9df = pd.read_csv(file_name)

10

11count_pairs=Counter()

12# write the code

13

14for date,group in df.groupby('Date'):

15products=group['Product']

16# products=group['Product'].unique()

17for pair in combinations(sorted(products),2):

18count_pairs[pair] +=1

19

20# Output the most frequent product pairs

21

22best_count= max(count_pairs.values())

23best_pair= [pair for pair,val in count_pairs.items() if val==best_count]

24for pair in best_pair:

25print(f"{pair[0]} and {pair[1]}: {best_count} times")

26

Average time0.015 s14.67 msMaximum time0.030 s30.00 ms1 out of 1 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 1Expected outputActual output

sales_data.csvsales_data.csv

Product A and Product B: 2 timesProduct A and Product B: 2 times

Product A and Product C: 2 timesProduct A and Product C: 2 times

TerminalTest cases

CODETANTRAHomeLearn Anywhere™

202501040046@mitaoe.ac.inSupportLogout

4.2.6. Titanic Dataset Analysis and Data Cleaning

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.

2. Display the last 5 rows of the dataset.

3. Get the shape of the dataset (number of rows and columns).

4. Get a summary of the dataset (using .info()).

5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().

6. Check for missing values and display the count of missing values for each column.

7. Fill missing values in the 'Age' column with the median age.

8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).

9. Drop the 'Cabin' column due to many missing values.

10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,0,5 2137,7.25,5
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2, 3101202,7.925,5
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,5
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,5
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,0
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,66,5
8,0,3,"Palsson, Master. Gosta Leonard",male,2,1,1,349909,21.075,5
9,1,1,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,5
10,1,2,"Nasser, Mrs. Nicholas (Adelie Achen)",female,14,1,0,237736,30.0708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

ExploreeTitanicDat...

1import pandas as pd

2import numpy as np

3

4# Load the Titanic dataset

5data = pd.read_csv('Titanic-Dataset.csv')

6

7print(data.head())

8

9

10print(data.tail())

11

12

13

14print(data.shape)

15

16print(data.info())

17

18

19print(data.describe())

20

21print(data.isnull().sum())

22

23

24data['Age'].fillna(data['Age'].median(), inplace=True)

25

26

Average timeMaximum time
0.113 s113.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected outputActual output
PassengerId Survived Pclass Fare Cabin Embarked PassengerId Survived Pclass Fare Cabin Embarked
01007.2500Nan501007.2500Nan5
120171.2833C85C120171.2833C85C
23017.9250Nan523017.9250Nan5

TerminalTest cases

CODETANTRAHomeLearn Anywhere™

202501040046@mitaoe.ac.inSupportLogout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.

2. Convert the 'Sex' column to numeric values (male: 0, female: 1).

3. One-hot encode the 'Embarked' column, dropping the first category.

4. Get the mean age of passengers.

5. Get the median fare of passengers.

6. Get the number of passengers by class.

7. Get the number of passengers by survival status.

8. Get the number of passengers by survival status.

9. Calculate the survival rate of passengers.

10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,0,5 2137,7.25,5
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2, 3101202,7.925,5
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,5
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,5
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,0
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,66,5
8,0,3,"Palsson, Master. Gosta Leonard",male,2,1,1,349909,21.075,5
9,1,1,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,5
10,1,2,"Nasser, Mrs. Nicholas (Adelie Achen)",female,14,1,0,237736,30.0708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

ExploreeTitanicDat...

1import pandas as pd

2import numpy as np

3

4# Load the Titanic dataset

5data = pd.read_csv('Titanic-Dataset.csv')

6data['FamilySize'] = data['SibSp'] + data['Parch']

7

8# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)

9data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)

10

11# 2. Convert 'Sex' to numeric (male: 0, female: 1)

12data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

13

14# 3. One-hot encode the 'Embarked' column

15data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

16

17# 4. Get the mean age of passengers

18print(data['Age'].mean())

19

20

21# 5. Get the median fare of passengers

22print(data['Fare'].median())

23

24

25# 6. Get the number of passengers by class

26print(data['Pclass'].value_counts())

27

28

Average timeMaximum time
0.041 s41.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected outputActual output
29.6991176470588229.69911764705882
14.454214.4542
34913491
12161216
21842184
Name: Pclass, dtype: int64Name: Pclass, dtype: int64

TerminalTest cases

CODETANTRAHomeLearn Anywhere™

202501040046@mitaoe.ac.inSupportLogout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.

2. Calculate the survival rate by embarkation location (Embarked_S).

3. Calculate the survival rate by family size (FamilySize).

4. Calculate the survival rate by being alone (IsAlone).

5. Get the average fare by passenger class (Pclass).

6. Get the average age by passenger class (Pclass).

7. Get the average age by survival status (Survived).

8. Get the average fare by survival status (Survived).

9. Get the number of survivors by class (Pclass).

10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,0,5 2137,7.25,5
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2, 3101202,7.925,5
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,5
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,5
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,0
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,66,5
8,0,3,"Palsson, Master. Gosta Leonard",male,2,1,1,349909,21.075,5
9,1,1,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,5
10,1,2,"Nasser, Mrs. Nicholas (Adelie Achen)",female,14,1,0,237736,30.0708,C

Note: Refer to the visible test case for better reference.

Sample Test Cases

ExploreeTitanicDat...

1import pandas as pd

2import numpy as np

3

4# Load the Titanic dataset

5data = pd.read_csv('Titanic-Dataset.csv')

6data['FamilySize'] = data['SibSp'] + data['Parch']

7data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)

8data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

9

10print(data.groupby('Pclass')['Survived'].mean())

11

12print(data.groupby('Embarked_S')['Survived'].mean())

13

14

15print(data.groupby('FamilySize')['Survived'].mean())

16

17print(data.groupby('IsAlone')['Survived'].mean())

18

19print(data.groupby('Pclass')['Fare'].mean())

20

21

22print(data.groupby('Pclass')['Age'].mean())

23

24

25print(data.groupby('Survived')['Age'].mean())

26

27

Average timeMaximum time
0.048 s48.00 ms

1 out of 1 shown test case(s) passed

Test case 1

Expected outputActual output
PclassPclass
10.62963010.629630
20.47282620.472826
30.24236330.242363
Name: Survived, dtype: float64Name: Survived, dtype: float64
Embarked_SEmbarked_S
10.62963010.629630
20.47282620.472826
30.24236330.242363

TerminalTest cases

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,0,0,330877,8.4583,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

titanicDat...

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8
9 print(data[data['Survived'] == 1]['Sex'].value_counts())
10
11 print(data[data['Survived'] == 0]['Sex'].value_counts())
12
13 print(data[data['Survived'] == 1]['Embarked_S'].value_counts())
14
15 print(data[data['Survived'] == 0]['Embarked_S'].value_counts())
16
17
18 children=data[data['Age'] < 18]
19 per=children['Survived'].mean()
20 print(per)
21 children['Survived'].mean()
22
23 adults=data[data['Age'] >= 18]['Survived'].mean()
24 print(adults)
25
26 print(data[data['Survived'] == 1]['Age'].median())
```

Average time

0.044 s

44.00 ms

Maximum time

0.044 s

44.00 ms

1 out of 1 shown test case(s) passed

Test case 1

44 ms

Debug

Test cases

Expected output

female....233

male.....109

Name: Sex, dtype: int64

male.....468

female.....81

Name: Sex, dtype: int64

+ ...

Actual output

female....233

male.....109

Name: Sex, dtype: int64

male.....468

female.....81

Name: Sex, dtype: int64

+ ...

Terminal

Test cases

< Prev Reset Submit Next >